



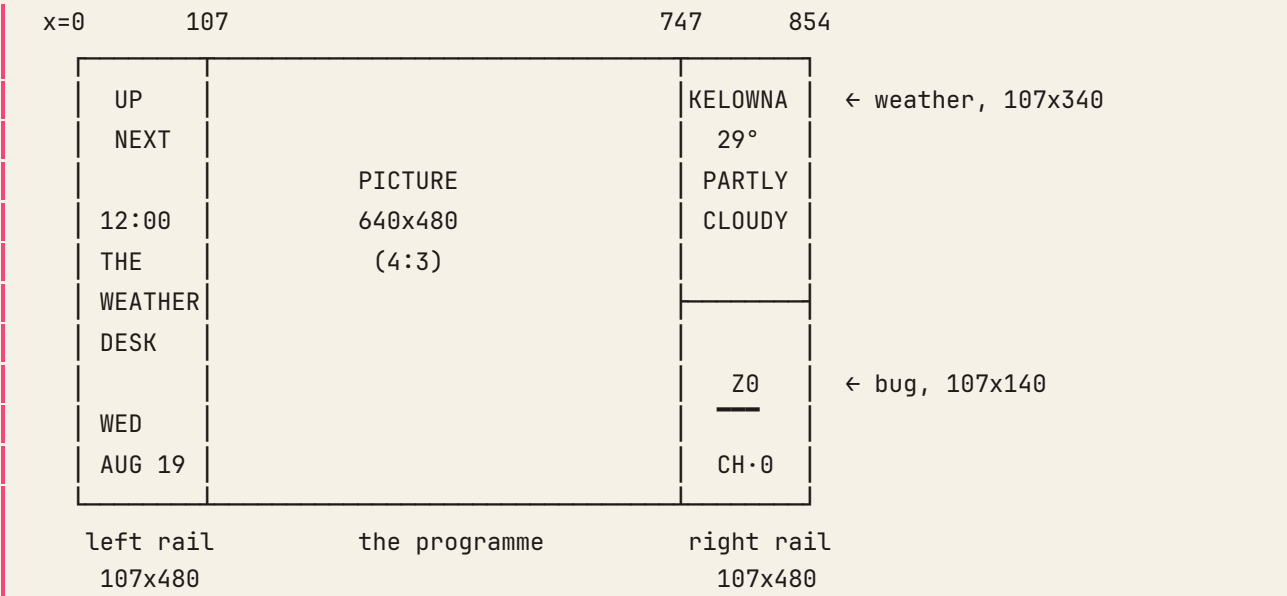
ON-AIR GRAPHICS – THE RAILS, THE BUG, THE WEATHER DESK

DOC NO. RL-Z0-WEATHER-A REV. A RIPOSTE LABORATORIES

Everything the channel puts on screen. Since the gutter rebuild, none of it is on top of the picture: the channel is 854x480, everything it airs is 4:3, so ffmpeg pillarboxes 640x480 of picture into the middle and leaves a 107-pixel black bar down each side. All the furniture lives in those bars.

CONTENTS

▶ The geometry lives in three numbers	3
▶ Does the gutter always exist?	4
▶ The weather desk	4
A music bed	5
▶ Things that will bite you	5
▶ Why there is no bottom strip any more	7
Measuring it yourself	9
▶ Changing the channel's resolution	9
▶ Wiring it into the schedule	10
▶ Performance	12



WHAT	WHERE IT COMES FROM	REFRESH
Left rail – UP NEXT header, date, station line	branding/z0-rail-left.png	every 15 min
Left rail – the listings themselves	ErsatzTV's own guide, via <code>epg_entries: 3</code>	per programme
Right rail top – conditions	branding/z0-rail-wx.png	every 15 min
Right rail bottom – the channel bug	branding/z0-rail-bug.png, made by <code>tools/make-bug.sh</code>	static
The Weather Desk (full screen, 4:3)	weather/z0-local-forecast.mp4, 2 min, 4×/day	hourly

Three consequences of being in the gutter rather than on the picture, all of them improvements and none of them cosmetic:

- ▶ **Everything is fully opaque.** The bug used to be dimmed to 72% and the weather card to 92%. That dimming existed only to let the programme show through; in a black bar it bought nothing and cost legibility.
- ▶ **Nothing has to get out of the way.** The "NEXT ." lower third used to fade up ten seconds into each programme and be gone by thirty. It now stays up for the whole item and lists **two** upcoming programmes with their start times, because a listing that is not covering anything has no reason to leave.
- ▶ **Everything is composited 1:1.** Each rail PNG is authored at exactly its final pixel size and every element sets `scale: false`, so no resample ever touches them. At a 107px column width, a resample is the difference between legible small type and mush – the old card was authored 920px wide and shrunk to 248.

SEC.01

THE GEOMETRY LIVES IN THREE NUMBERS

`Z0_FRAME_W` (854), `Z0_FRAME_H` (480) and `Z0_CONTENT_W` (640). Every generator derives the rail width from them – `RAIL_W = (FRAME_W - CONTENT_W) / 2` – and nothing writes 107 down. The right rail's two blocks are sized so they abut exactly (`WX_H` 340 + `BUG_H` 140 = 480), which is what makes the red spine down its inner edge read as one unbroken rule instead of two stubs.

The usable text column is the same in both rails: `RAIL_W - SPINE - 2*PAD`, i.e. $107 - 3 - 16 = 88\text{px}$. The element YAMLS quote it as a percentage of frame width (10.3), and `tools/z0-weather.py` lays both rails out with a running cursor rather than absolute coordinates, so adding a line cannot silently push another one on top of the block below.

SEC.02

DOES THE GUTTER ALWAYS EXIST?

Yes, for everything this channel actually airs. Of the 562 items in `/media`, **556 are exactly 640x480**. The five that are not are 720x480 NTSC newsreels – SAR 8:9, DAR 4:3 – and ErsatzTV’s `SquarePixelFormatSize` resolves anamorphic before it pads, so they land at 640x480 too. The only genuinely 16:9 file was the station’s own weather segment, and that is now rendered 4:3 (1440x1080) so the picture window never changes size.

If a real 16:9 item is ever added, it fills the frame edge to edge and there is **no gutter** – the rails would then sit on the picture. That is the one thing to check before adding widescreen content.

SEC.03

THE WEATHER DESK

`tools/z0-weather.sh` fetches from **Open-Meteo** (no key, no account, so nothing to rotate or expire), renders with the `ffmpeg` inside the ErsatzTV image, and installs each asset atomically. `tools/z0-weather.py` does the fetching and lays out the `drawtext` calls; the shell script drives `ffmpeg`.

BASH

```
tools/z0-weather.sh          # both rails + full-screen segment
tools/z0-weather.sh --card-only # skip the segment (the slow part)
```

Location comes from the environment, defaulting to Kelowna, BC:

BASH

```
Z0_WX_LAT=49.88307 Z0_WX_LON=-119.48568 Z0_WX_CITY=KELOWNA Z0_WX_REGION=BC \
Z0_WX_TZ=America/Vancouver tools/z0-weather.sh
```

Two cron entries on the playout host, because the two halves have very different costs – the rails are seconds, the segment is most of a minute of encoding that competes with the live transcode:

```
* /15 * * * * tools/z0-weather.sh --card-only
5 * * * * tools/z0-weather.sh
```

A MUSIC BED

The segment ships with a quiet low drone. Drop any audio file into `weather/bed/` and the next render uses it instead, looped and faded. `Z0_SILENT=1` gives silence.

SEC.04

THINGS THAT WILL BITE YOU

Write atomically or the overlay disappears for a whole programme. ErsatzTV opens these files while streaming. If it catches a half-written PNG, `ImageElement` logs `Failed to initialize image element; will disable for this content` and the graphic is gone until the next item – a silent, self-healing failure that looks like a config mistake. Every install here is `write-temp + rename(2)`.

The segment's duration must never change. ErsatzTV scheduled it from the duration it saw at scan time. The script pins the output to `Z0_WX_SEGMENT_SECS` and **refuses to install** a file of the wrong length, so the library row stays valid. Keep one filename. Its **resolution** is not protected the same way – see "Changing the channel's resolution" below.

Overlays refresh per programme, not per frame. ErsatzTV re-reads an image or text template when each item's ffmpeg process starts. During a 90-minute film the conditions are whatever was on disk when that film began – which is why the weather rail carries an `AS OF` stamp instead of pretending to be live. The segmenter also works ahead, so what is on air is a little behind the file on disk.

The whole element YAML is a scriban template, so a template error costs you the element, not the field. GraphicsElementLoader renders the entire file through scriban **before** parsing it as YAML. A bad expression anywhere in the file logs exactly two lines –

```
[WRN] Failed to render graphics element YAML definition as scriban template
[WRN] Failed to load text graphics element from file ...; ignoring
```

– and the element is simply absent from air while every other element renders perfectly. Nothing appears in the frame to tell you which one broke.

Format times with `format_datetime`, not scriban's `date.to_string`. The latter fails on `Epg[n].Start` and takes the element out exactly as above. The graphics engine imports `format_datetime(value, timeZoneId, format)` and `convert_timezone(value, timeZoneId)`; use them. Naming the zone is the point – `Start` is a `DateTimeOffset`, and anything that formats it implicitly formats it in the **container's** timezone, which is right today by luck rather than by design.

A style span cannot cross a line break. Inline styles are matched with `\[(\w+)\](.*?)\[/\1\]` and `.` does not match a newline, so open and close `[t]...[/t]` on the same line.

Skia does not use `fontconfig`. Text elements render through SkiaSharp/RichTextKit, which ignores `fontconfig` entirely – `fc-list` and `fc-match` finding a face inside the container proves nothing. Without `include_fonts_from` the log says `Could not find font ...; using default` and the listings quietly render in the wrong face. The two faces we need are staged in `branding/fonts/`.

`location_x` / `location_y` exist on `ImageGraphicsElement` and are never read. `ImageElement.InitializeAsync` does not pass them to `LoadImage`, so setting them does nothing at all – silently. Position with `location` (one of `TopLeft`, `TopRight`, `BottomLeft`, `BottomRight`, `TopMiddle`, `BottomMiddle`, `LeftMiddle`, `RightMiddle`, `MiddleCenter`) plus the margin percentages. Beware `CenterLeft` / `CenterRight` / `TopCenter` / `BottomCenter`: those names appear in the binary but belong to `ErsatzTV.Core/Next`, the Rust rewrite's model, not to the `WatermarkLocation` enum this engine uses.

`place_within_source_content: true` does the opposite of what the rails want. It offsets margins by half the padding so an element sits inside the **picture**. The rails want the frame, which is the default (`false`).

`drawtext` is missing from some ErsatzTV images. The old 25.2 `-nvidia` image was built without it despite advertising freetype and fontconfig, so the `make-*.sh` generators could not run against it. The 26.7.1 image has it.

SEC.05

WHY THERE IS NO BOTTOM STRIP ANY MORE

The channel used to carry a paged information strip along the bottom of the picture, as a **subtitle** graphics element. It is gone, and its content moved into the rails: conditions to the right, listings to the left. This is the single most important thing on this page, because the strip took the channel off air 25 times in one morning.

ErsatzTV composites graphics itself, in-process, and pipes one full-frame RGBA stream into ffmpeg as input `[1:0]`. Image and text elements are rasterised once and reused, so they are nearly free. **A subtitle element is re-rendered every single frame whether or not its content changed**, and there is no static-content fast path. ErsatzTV then logs

```
[FTL] Media item [N] on channel 0 transcoded at 0.254x (NOT throttled)
      which is NOT fast enough to support playback
```

abandons the item and cuts to **fallback colour bars**, which carry no graphics at **all** – so the symptom is "the overlays vanished", not "the channel is slow", and the guide, the DB and `/api/status` all still look perfect.

The clean natural experiment: item 2513 carried `{up-next, bug, strip}` and ran 0.258x; item 2291 carried `{up-next, card, bug}` and ran 1.28x. **Same number of elements** – so it was the subtitle element specifically, not the overlay count.

Three things it was **not**, each measured rather than assumed:

- ▶ **Not libass.** ffmpeg renders the exact same `.ass` at **635 fps** (21x realtime). The subtitle rasteriser was never the bottleneck.
- ▶ **Not the box.** 12 cores at load 2.2, no cgroup CPU limit, ErsatzTV pinned at ~238% – it had headroom and still underran. The cost is serialised inside one process, so more cores will not fix it.

-
- **Not animation.** The strip was rewritten from a scrolling `\move` crawl to static paged text, so every frame within a page was byte-identical. It made **no difference at all** – still 0.254x.

The only lever that worked was **resolution**, because the per-frame cost is proportional to pixel count: 1920x1080 → 854x480 → 640x480. That is why the channel was SD, and why 854x480 had to be abandoned – it left the Prelinger block at 0.996x, still logging the odd [FTL].

Removing the strip is what paid for the gutter. A 107px column cannot hold a horizontal crawl anyway, so the redesign and the performance fix are the same change. Measured on Lilac Time (1928), the heaviest title on the channel, via `/api/troubleshoot/playback.m3u8` (which runs with `HlsRealtime: false`, so there is no `-readrate` cap and the number is real capability):

FRAME	ELEMENTS	SPEED
640x480	old set, with the strip	1.28x
854x480	old set, with the strip	1.01x ← why 854x480 was abandoned
854x480	the rails (3 image + 1 text)	2.42x
640x480	the rails	2.54x
854x480	no graphics at all	5.20x

So the channel went **up** a third in pixel count and still nearly doubled its headroom. Note the last two rows: with the subtitle element gone, 33% more pixels costs only 5%, because the pipeline is no longer graphics-bound.

Speed reports mean what they say. Treat anything under ~1.2x as a channel that will fall over eventually, and confirm with whether **fallback filler actually starts** – `grep -ci colorbars` over `/config/logs/ersatztv<date>.log`. Judge by [FTL], not [WRN]: the warning threshold is 1.5x when unthrottled and is miscalibrated against the 1.05x cap ErsatzTV applies to normal playout items.

MEASURING IT YOURSELF

```
GET /api/troubleshoot/playback.m3u8
    ?mediaItem=<id>&channel=0&ffmpegProfile=<id>&streamingMode=4
    &graphicsElement=<id>&graphicsElement=<id>...
```

`channel=0` selects media-item mode, which takes an explicit profile and an explicit element list – exactly an A/B harness. (`channel=<n>` is a different mode that replays the real playout and **requires** a `start` parameter; without it you get a bare 404.) Do this on the `z0-screener` replica, never on the live instance.

SEC.06

CHANGING THE CHANNEL'S RESOLUTION

Fewer moving parts than it used to have – the ASS coordinate space is gone with the strip, and nothing is scaled by a percentage of frame width any more.

1. **FFmpegProfile 1** – `ResolutionId`, `VideoBitrate`, `VideoBufferSize`. There is no stock 16:9 480p row, so `854x480` is `Resolution` id 5, added by hand; `640x480` is the stock id 0.
2. **Re-render every rail asset** with the new geometry, because they are composited 1:1 and pixel-exact: `bash Z0_FRAME_W=... Z0_FRAME_H=... Z0_CONTENT_W=... tools/z0-weather.sh Z0_FRAME_W=... Z0_FRAME_H=... Z0_CONTENT_W=... tools/make-bug.sh`
3. **width_percent** in `z0-upnext.yml` and the two margin percentages, which are the only percentages left. They encode the 88px column and the 8px inset as fractions of frame width.
4. **The weather segment's aspect**, if the channel stops being 4:3. It is rendered at `Z0_SLIDE_WxZ0_SLIDE_H` (1440x1080) so the picture window never changes size when the forecast comes on.

Then restart `z0-ersatztv`, then `z0-uplink` – Owncast is single-quality passthrough and cannot carry a mid-stream resolution change, so the tower drops for ~15 s and the uplink reconnects itself.

Changing the weather segment's resolution needs the library row updated. ErsatzTV

builds the scale/pad filter from the `MediaVersion` row it recorded at scan time, not from the file. Re-rendering the segment at a new size and leaving the row alone makes ffmpeg scale 1440x1080 to the dimensions of the **old** 1920x1080 – the desk goes out horizontally stretched, and nothing warns. Either let the app rescans (`UPDATE LibraryPath SET LastScan = NULL WHERE Path='/media'` then restart) or update `MediaVersion Width/Height/SampleAspectRatio/ DisplayAspectRatio` directly, which is what this change did.

SEC.07

WIRING IT INTO THE SCHEDULE

The element set lives in `playout/_sequences.yml`, in the `graphics_up` sequence. That matters operationally: `_sequences.yml` is an import fragment, so the element set can be deployed on its own without redeploying `channel-z0.yml`, whose day rotation has to be got right or the channel airs the wrong day's programming.

`tools/wire-graphics.py` rewrites `playout/channel-z0.yml`, inserting the `graphics_on/graphics_off` instructions and the four daily forecast slots across all seven day blocks. It is idempotent – re-run it after editing the schedule.

BASH

```
tools/wire-graphics.py playout/channel-z0.yml playout/channel-z0.yml
```

It also repairs two things that stop the playout building on 26.x, both of which fail **silently**:

- ▶ **search: must be null.** This file used to write `search: Cartoons`, using the slot as a label. From 25.4 the schedule is validated against a JSON schema where `search` is `{"type": "null"}` and every object is `additionalProperties: false`. A failed validation means `SequentialPlayoutBuilder` returns `None` and the playout does not build at all. The label now lives in a comment, where it always belonged.
- ▶ **Every `pad_until` needs an explicit `tomorrow`.** 26.x changed `Tomorrow` from `bool` to `string` so it can hold an expression, and `YamlPlayoutPadUntilHandler` does

new Expression(padUntil.Tomorrow) with no null check. NCalc throws ExpressionString cannot be null or empty and the whole build dies. The branch only runs when the pad target is already in the past, so a bare pad_until is a landmine that detonates the first time the playout is rebuilt later in the day than its earliest morning block.

There is no strip_down / strip_up any more; with nothing expensive left to switch off, all four elements stay up for everything. A sequence: naming a key that does not exist is a **silent no-op** in ErsatzTV (ExecuteSequence filters the definition and iterates an empty list), which is what lets an older deployed channel-z0.yml that still calls strip_down keep building correctly against the new fragment.

Graphics elements are attached to playout items as they are built, so existing items never gain a newly-added element. The options, cheapest first: insert the rows directly into PlayoutItemGraphicsElement for the items already built; cut the playout at a block boundary with tools/z0-day-align.py; or reset it outright (delete PlayoutItem, PlayoutAnchor, PlayoutHistory) – and a reset re-enters the week at block one, so it must be done on the weekday the file is rotated to or the channel airs the wrong day.

Element definitions must exist and be **registered** before the build that references them. ErsatzTV refreshes them at startup, hourly, and when a stream session starts – but at startup it builds playouts **before** refreshing, so a brand-new element file needs one restart to register.

SEC.08

PERFORMANCE

Compositing is CPU work (SkiaSharp), and the blend is a GPU `overlay_cuda`. The channel encodes with `NVENC` (`HardwareAcceleration=2` on the `ffmpeg` profile); the GTX 1650 was otherwise idle while the CPU did all of it. `NVENC` was necessary but **not sufficient** – it fixed the encode, not the compositor upstream of it, and the channel still fell to colour bars while the subtitle strip was on.

The channel runs at **854x480 / 1600k**. Rule of thumb: image and text elements are effectively free, a subtitle element costs about 4x realtime at 1080p, and that price scales with pixel count.

SOURCE docs/weather.tex SHA 4423d0b8abad BUILT 2026-09-07